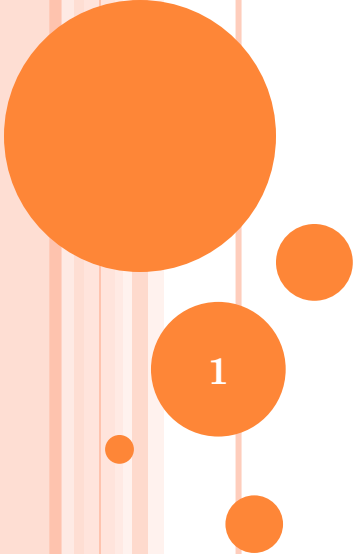




SISTEME DE OPERARE

Curs 5: Gestiunea proceselor

Kinga Márton

Universitatea Tehnică din Cluj-Napoca
Departamentul de Calculatoare

GESTIUNEA PROCESELOR

- În cuprins

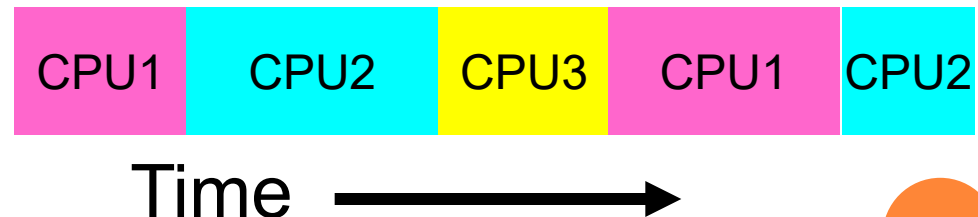
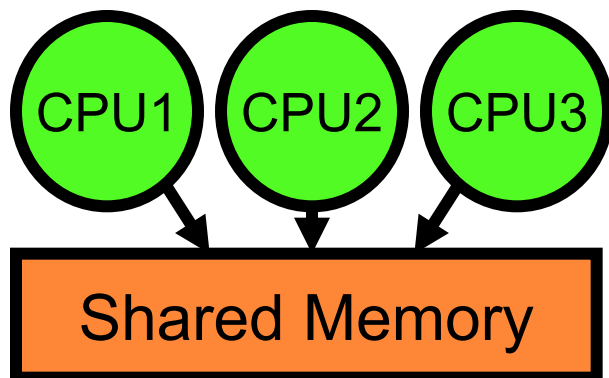
- **Concepte generale** legate de gestiunea proceselor
 - Reprezentarea și controlul proceselor
 - Stările și comportamentul proceselor
 - Structuri de date pentru gestiunea proceselor
 - Utilizarea structurilor pentru a controla execuția proceselor
- **Apeluri sistem Linux** pentru gestiunea proceselor

PROCESUL

- Scopul SO
 - Rularea programelor și aplicațiilor
 - Concurența pentru utilizarea eficientă a resurselor
 - Timp de răspuns mic
- Gestiunea concurenței este o sarcină grea
 - partajarea resurselor în mod corect + protecție
 - Multiprogramarea
 - permite efectuarea mai multor operații în același timp
 - EX. PROCESARE ȘI I/E
 - Concurența
 - În sisteme monoprocesor – time sharing sau multitasking
 - PSEUDOPARALELISM
 - Sisteme multiprocesor
 - PARALELISM REAL
- Soluție: **procesul secvențial** - concept central în SO
 - modelul conceptual care facilitează înțelegerea și gestiunea paralelismului
 - Flux de execuție secvențial într-un spațiu de adrese separat
 - Abstracția unui **program în execuție**
 - Unitatea de activitate caracterizată prin
 - Secvența de instrucțiuni care trebuie executată
 - Starea curentă
 - Resursele sistem asociate

TOTUL ESTE PROCES (APROAPE)

- Toate aplicațiile care pot fi rulate și chiar și SO-ul
 - Sunt procese
- Sistemul poate rula aparent simultan zeci sau chiar sute de procese
 - Furnizează ușurința în folosirea sistemului
- Conceptual fiecare proces are propriul **procesor virtual**
 - SO gestionează concurența prin abstracția de mașină virtuală
 - Cum creăm iluzia că avem mai multe procesoare?
 - Dacă de fapt avem un număr restrâns sau chiar numai un singur procesor
 - Soluție: **Multiplexare în timp** – **time sharing**
 - Procesorul comută foarte rapid între procese



SEPARAREA MECANISMELOR DE POLITICI

- Implementarea (adecvată) a virtualizării procesorului necesită
 - **Mecanisme** de nivel coborât
 - Implementează o anumită **funcționalitate**
 - Ex. schimbarea de context prin mecanismul de time sharing
 - **Politici** - inteligența de nivel înalt
 - Algoritmi pe baza cărora se iau anumite **decizii** în cadrul SO
 - Ex. politica de planificare – din mai multe procese existente care va fi rulat

CONTEXTUL PROCESULUI

- Procesul este caracterizat de părțile din sistem pe care
 - le accesează (citește)
 - le afectează (actualizează)în timpul execuției

- **Memoria** care poate fi accesată de proces – **spațiul de adrese**

- Instrucțiunile programului sunt în memorie,
- Datele citite și actualizate sunt în memorie

- **Regiștrii speciali**

- PC – Program Counter (IP – Instruction Pointer)
 - Care instrucțiune din program se va executa în pasul următor
- SP – Stack Pointer
 - Gestiunea parametrilor funcțiilor, variabile locale, adrese de retur, etc.

- **Informații de I/E**

- Fișiere deschise, etc.

PROCESE ȘI PROGRAME

- Diferență subtilă, dar crucială

- **Procesul**

- Entitate **dinamică** – este o activitate
 - Instanța unui program în execuție
- Două componente fundamentale
 - Flux secvențial de instrucțiuni
 - Care poate fi partajat cu alte procese care execută același program
 - Set de date asociate programului executat
 - Toate resursele cu care interacționează procesul
- Ex. folosesc *gedit/vim* pentru a edita un fișier de comenzi și pentru a scrie cod C
 - Același program, două procese diferite

- **Programul**

- Entitate **statică** (inactivă)
 - Parte din starea procesului
- Un program poate invoca mai multe procese

PROCESE ȘI PROGRAME

○ Analogia cu gătitul

- Programul – un set de instrucțiuni care reprezintă un algoritm pentru a rezolva ceva
 - **Rețeta** unei prăjituri
- Date de intrare
 - **Ingredientele** (făină, ulei, ouă, lapte, zahăr, etc.)
- Date de ieșire



PROCESE ȘI PROGRAME

○ Analogia cu gătitul

- Programul – un set de instrucțiuni care reprezintă un algoritm pentru a rezolva ceva
 - **Rețeta** unei prăjituri
- Date de intrare
 - **Ingredientele** (făină, ulei, ouă, lapte, zahăr, etc.)
- Date de ieșire



PROCESE ȘI PROGRAME

○ Analogia cu gătitul

- Programul – un set de instrucțiuni care reprezintă un algoritm pentru a rezolva ceva
 - **Rețeta** unei prăjituri
- Date de intrare
 - **Ingredientele** (făină, ulei, ouă, lapte, zahăr, etc.)
- Date de ieșire
- Procesul - efectuarea instrucțiunilor pas cu pas, sau executarea programului
 - Activitatea **gătitului**
- Procesorul
 - **Bucătarul**
- Mai multe procese efectuează același program
 - Mai multe torturi pe baza aceleiași rețete
- Întrerupere
 - Se rănește la deget apelează la trusa de prim ajutor
 - Ghidul din trusă e codul întreruperii
- Planificare
- Priorități



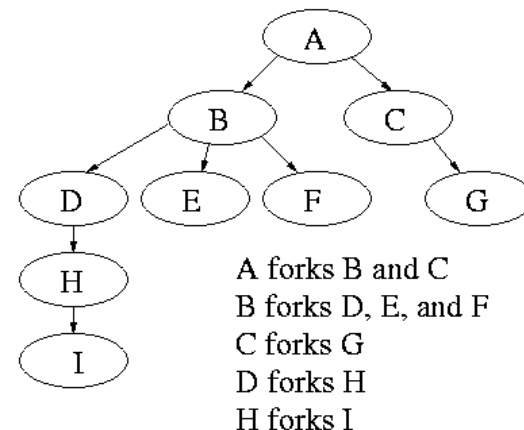
CREAREA ȘI TERMINAREA UNUI PROCES

- Principalele evenimente care duc la **crearea** unui proces
 - **Inițializarea sistemului** (foreground – prim plan, background - fundal, daemon)
 - Un **proces invocă** un apel sistem pentru crearea unui nou proces (procese înrudite dar independente)
 - **Utilizatorul cere** crearea unui nou proces (comandă, clic pe iconiță)
 - Inițierea unui job în sisteme batch (mainframe)
 - În fiecare caz **un proces existent cere crearea unui nou proces**
 - Apelul sistem **fork** / CreateProcess
-
- Condiții care duc la **terminarea** unui proces
 - **Ieșire normală, voluntară** (transmite la SO că a terminat)
 - **Ieșire cu eroare, voluntară** (ex. gcc suma.c, dar fișierul nu există)
 - **Eroare fatală, ieșire involuntară** (în general datorită unui bug)
 - **Terminat de un alt proces, ieșire involuntară** (kill)
 - Apelurile sistem **exit** / ExitProcess și **kill** / TerminateProcess

IERARHII DE PROCESE

○ Relație părinte – fiu

- Un proces are un singur părinte
- Un proces poate avea zero sau mai mulți fii



○ Grup de procese

- Procesul și toți descendenții săi formează un grup

○ Ierarhie

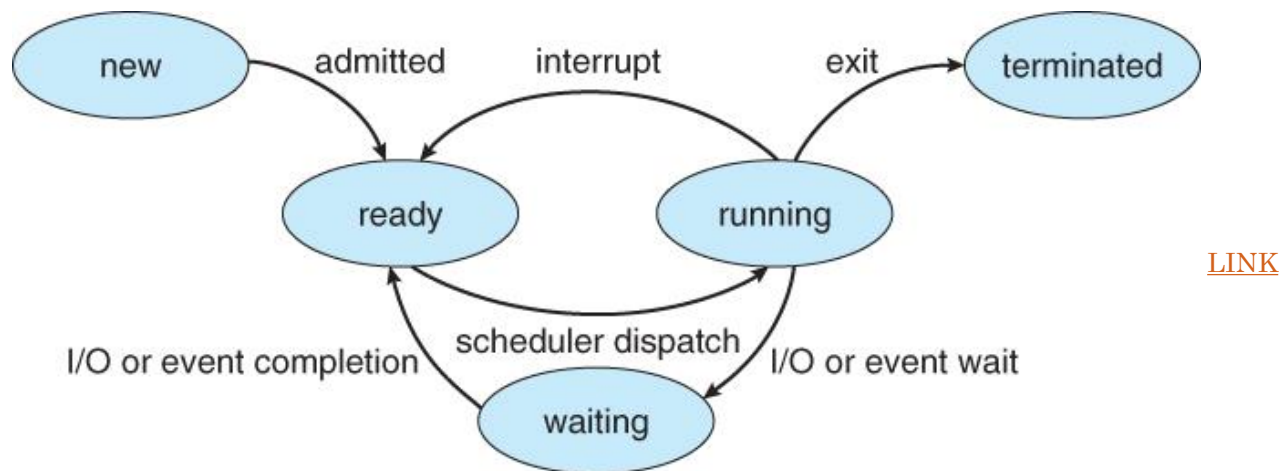
- Toate procesele aparțin unei singure ierarhii de procese
 - Rădăcina este procesul **init**

○ În Windows

- Există relația părinte – fiu
- Nu există o ierarhie
 - Toate procesele sunt egale (aprox.)
 - Handle - jeton
 - Folosit pentru controlul procesului fiu
 - Poate fi predat altui proces

STĂRILE PROCESULUI

○ Model cu cinci stări

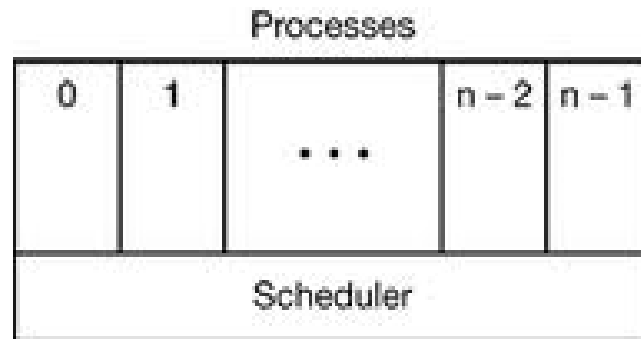


○ Procesul trece prin mai multe stări pe durata ciclului de viață

- **new**: proces nou creat
- **ready**: procesul e gata de rulare, așteaptă să fie planificat
- **running**: procesul e în execuție
- **blocked**: procesul este blocat în așteptarea unui eveniment extern
- **terminated**: procesul și-a terminat execuția

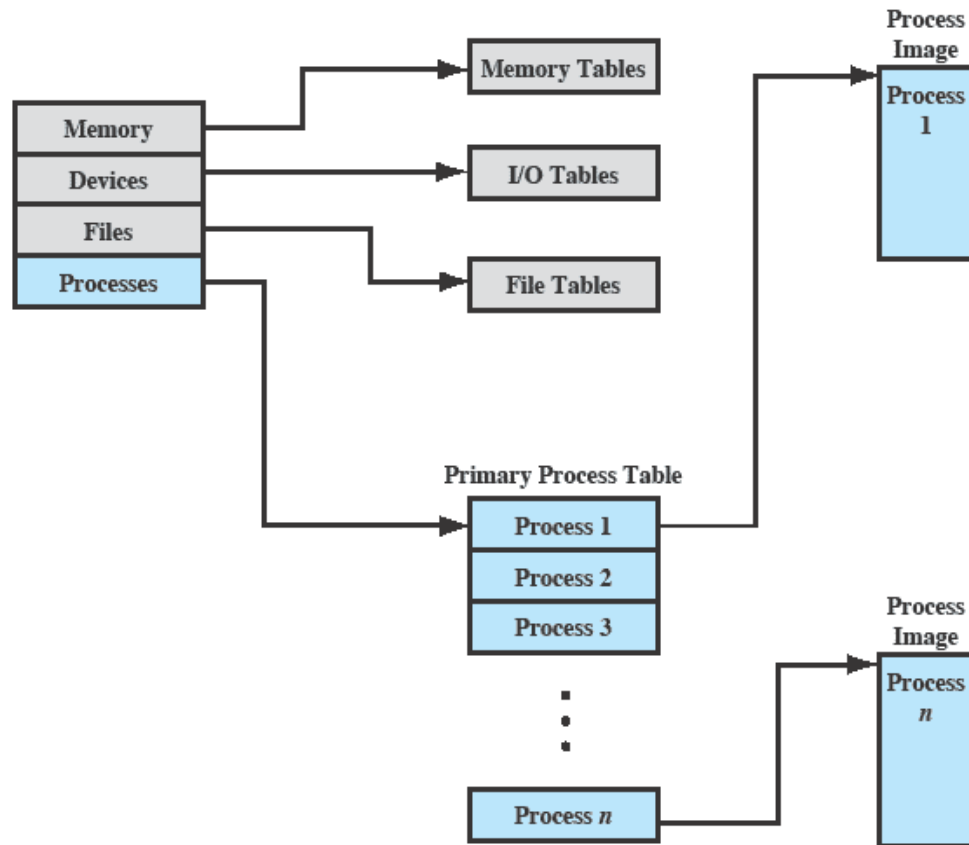
MODELUL SO STRUCTURAT PE PROCESE

- La nivelul de bază se află **planificatorul**
 - Gestionează întreruperile
 - Planifică execuția proceselor
 - Ascunde complexitatea opririi și pornirii proceselor



Tanenbaum, pg. 79

TABELE DE CONTROL



Stallings, pg. 127

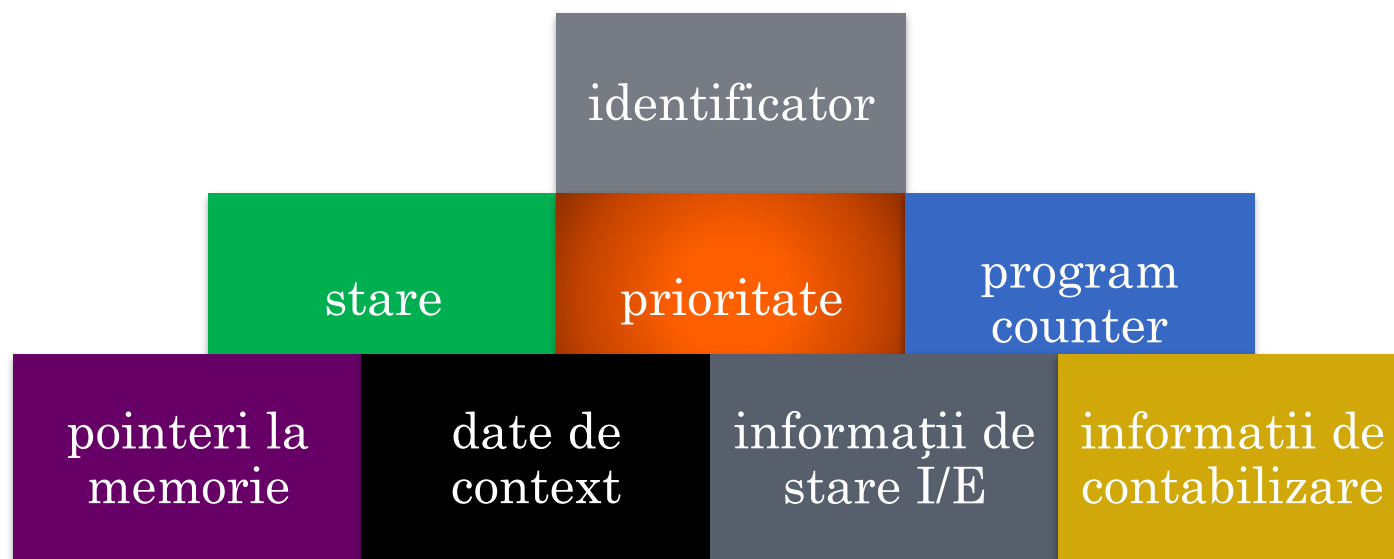
Figure 3.11 General Structure of Operating System Control Tables

- SO este un program sistem -> are nevoie de structuri de date pt. a gestiona resursele

IDENTIFICAREA PROCESULUI

○ Procesul

- În mod formal este un **flux secvențial de execuție în spațiul de adrese propriu**
- Include
 - Fluxul secvențial – program, regiștrii, etc.
 - Resurse protejate – conținutul spațiului de adrese, starea i/e (descriptori de fișier, etc.)
- Caracterizat de elemente cum ar fi:



IMPLEMENTAREA PROCESELOR

- Tabela proceselor
 - Vector de structuri – **Blocuri de control al proceselor (BCP)**
- Fiecare proces are o intrare în această tabelă
 - Conține attributele de control ale procesului
 - Include
 - **Identificatorul** unic de proces
 - Index în tabela proceselor (în unele implementări)
 - Informații pentru **controlul** procesului
 - Prioritate, planificare, alocarea memoriei, drepturi și utilizarea resurselor
 - Informații de **stare** a procesorului
 - Regiștri de stare și control (PC, SP)
 - Regiștri utilizator
- Toate informațiile care trebuie salvate când procesul este pus în așteptare (running -> ready sau ready -> blocked) astfel încât să poată fi reluat ulterior ca și când nu ar fi fost oprit

IMAGINEA PROCESELOR ÎN MEMORIA VIRTUALĂ

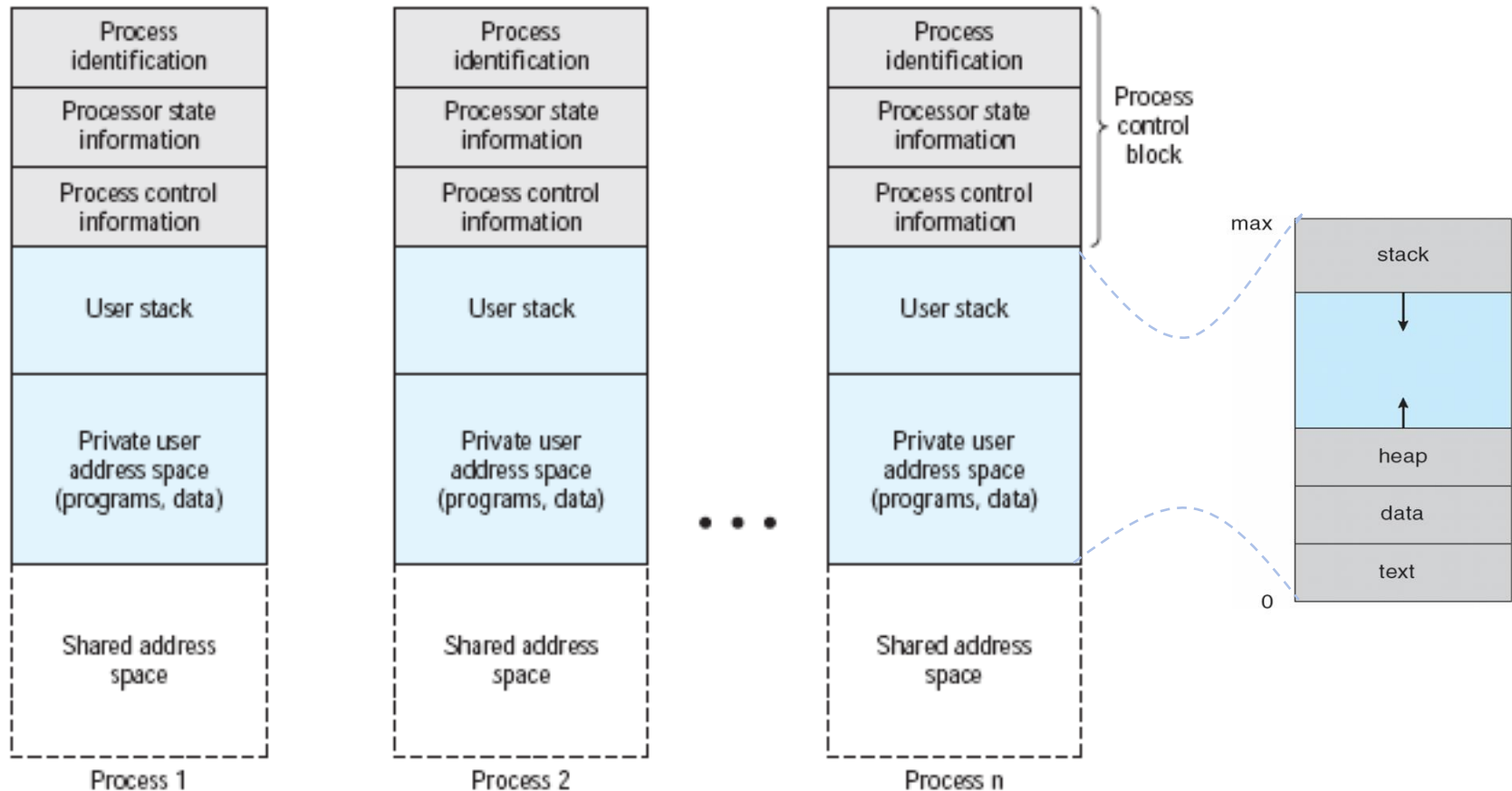
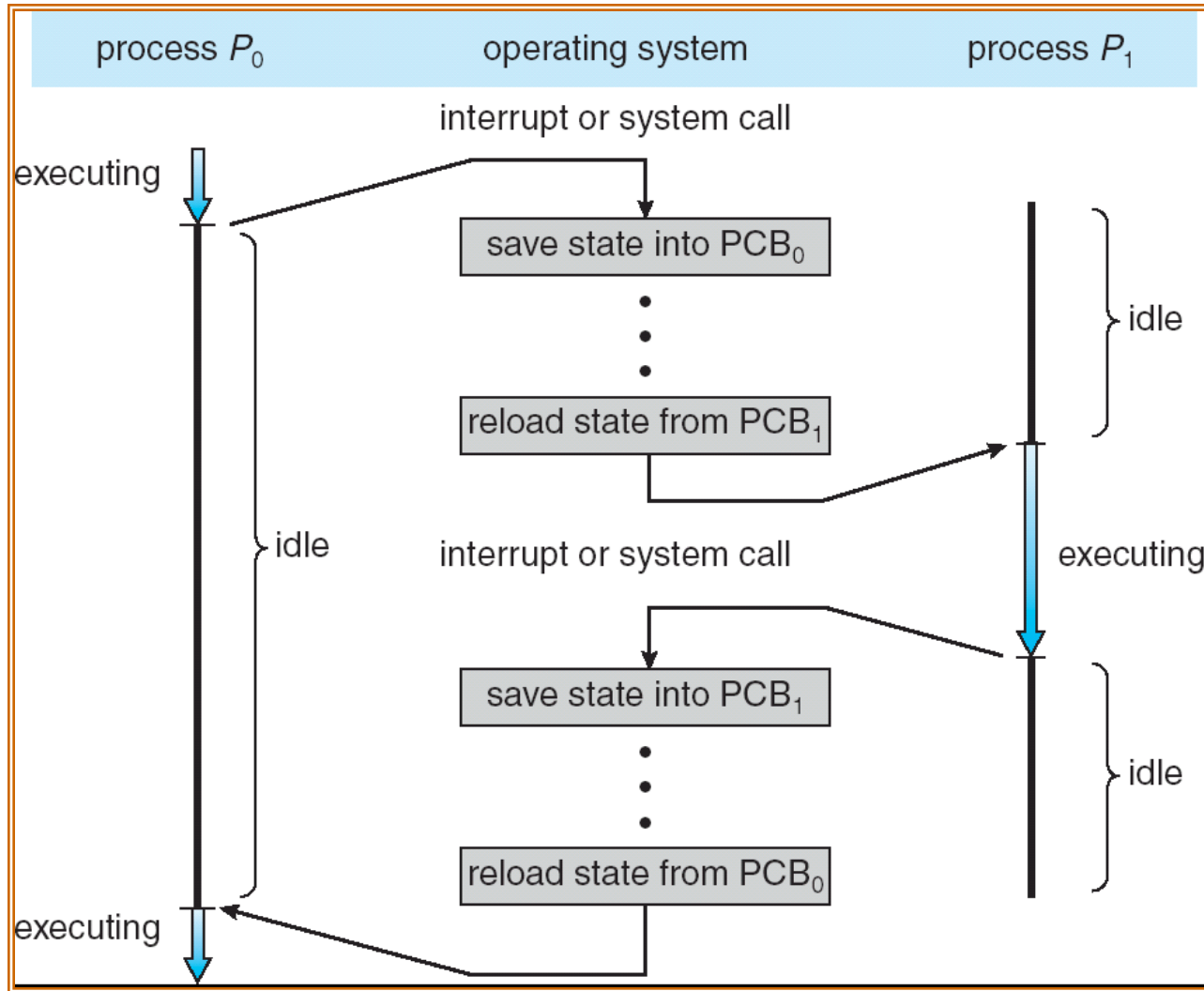
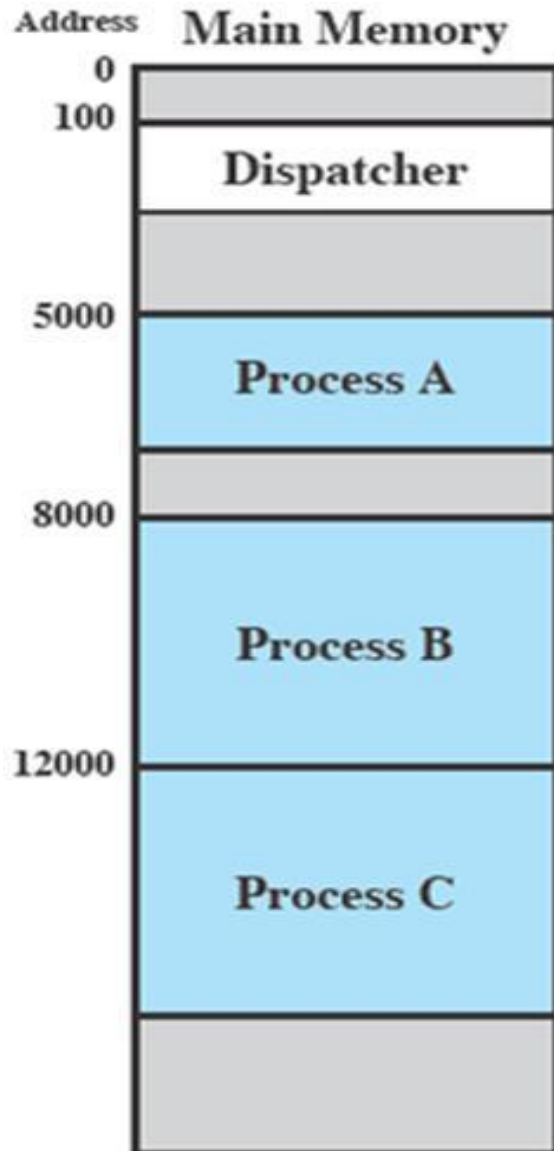


Figure 3.13 User Processes in Virtual Memory

SCHIMBARE DE CONTEXT



TRASAREA EXECUȚIEI



- Trei procese: A, B și C încărcate în memorie – din punctul de vedere al memoriei

5000	8000	12000
5001	8001	12001
5002	8002	12002
5003	8003	12003
5004		12004
5005		12005
5006		12006
5007		12007
5008		12008
5009		12009
5010		12010
5011		12011

(a) Trace of Process A

(b) Trace of Process B

(c) Trace of Process C

5000 = Starting address of program of Process A
 8000 = Starting address of program of Process B
 12000 = Starting address of program of Process C

Stallings, pg. 111, 112

Figure 3.3 Traces of Processes of Figure 3.2

TRASAREA EXECUȚIEI CONCURRENTE

1	5000				
2	5001				
3	5002				
4	5003				
5	5004				
6	5005				
----- Timeout					
7	100				
8	101				
9	102				
10	103				
11	104				
12	105				
13	8000				
14	8001				
15	8002				
16	8003				
----- I/O Request					
17	100				
18	101				
19	102				
20	103				
21	104				
22	105				
23	12000				
24	12001				
25	12002				
26	12003				
27	12004				
28	12005				
----- Timeout					
29	100				
30	101				
31	102				
32	103				
33	104				
34	105				
35	5006				
36	5007				
37	5008				
38	5009				
39	5010				
40	5011				
----- Timeout					
41	100				
42	101				
43	102				
44	103				
45	104				
46	105				
47	12006				
48	12007				
49	12008				
50	12009				
51	12010				
52	12011				
----- Timeout					

- din punctul de vedere al procesorului

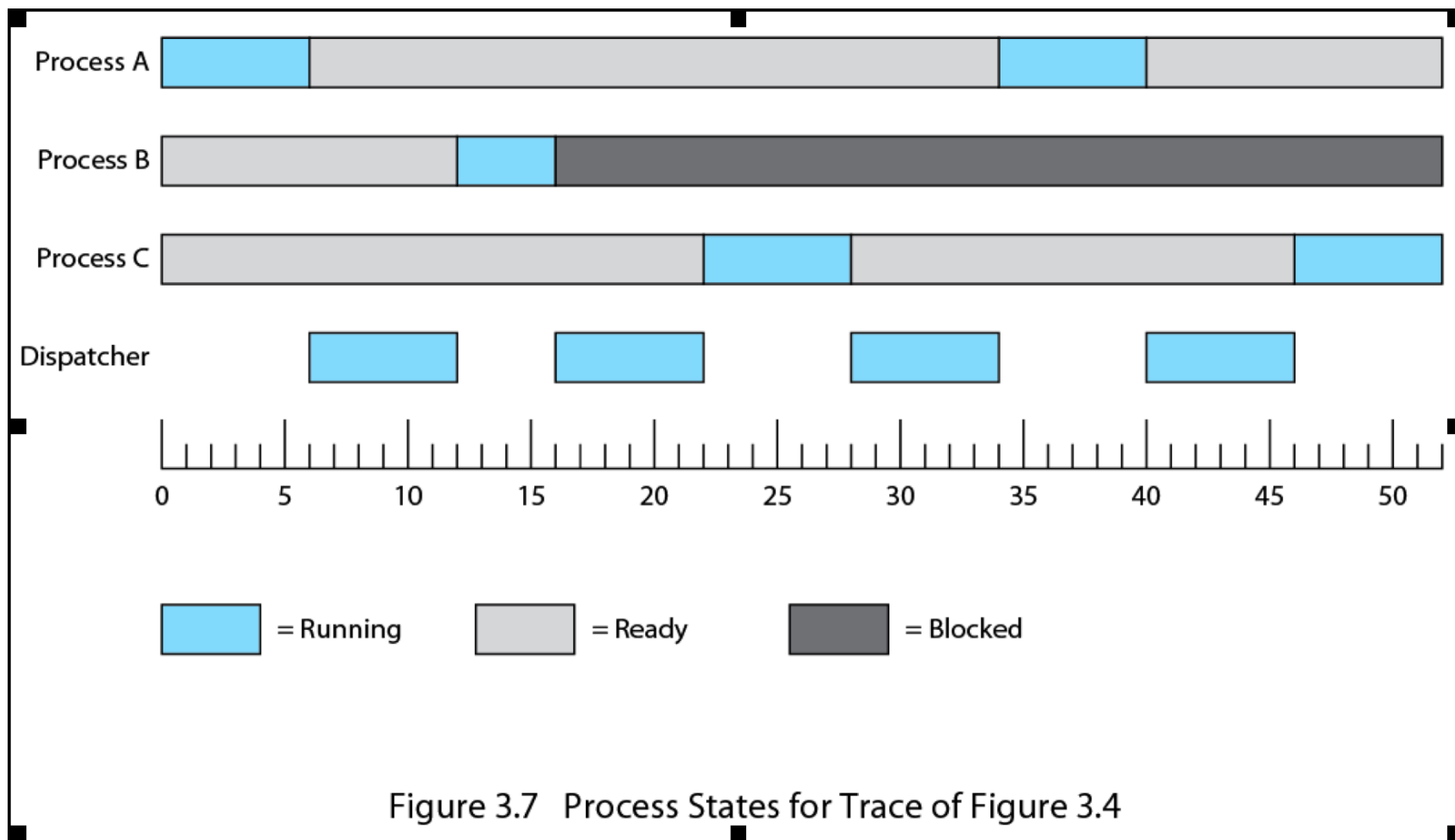
Stallings, pg. 113

100 = Starting address of dispatcher program

Shaded areas indicate execution of dispatcher process;
first and third columns count instruction cycles;
second and fourth columns show address of instruction being executed

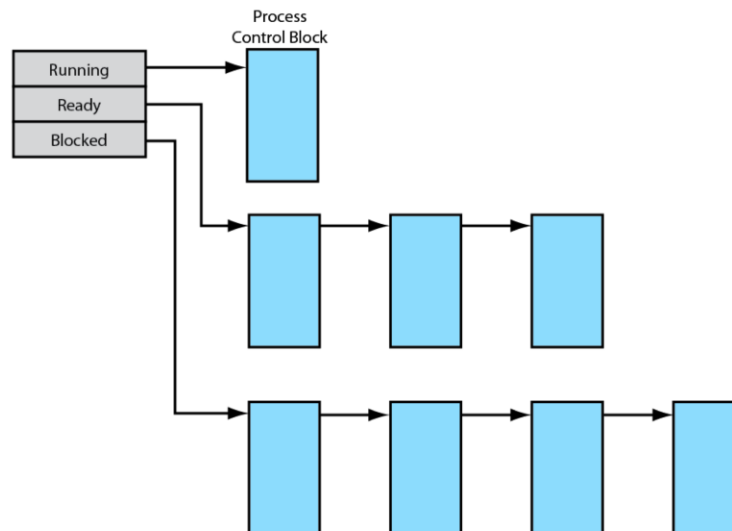
Figure 3.4 Combined Trace of Processes of Figure 3.2

STĂRILE PROCESELOR PENTRU TRASĂRILE PRECEDENTE



MULTIPLEXARE ÎNTRE PROCESE

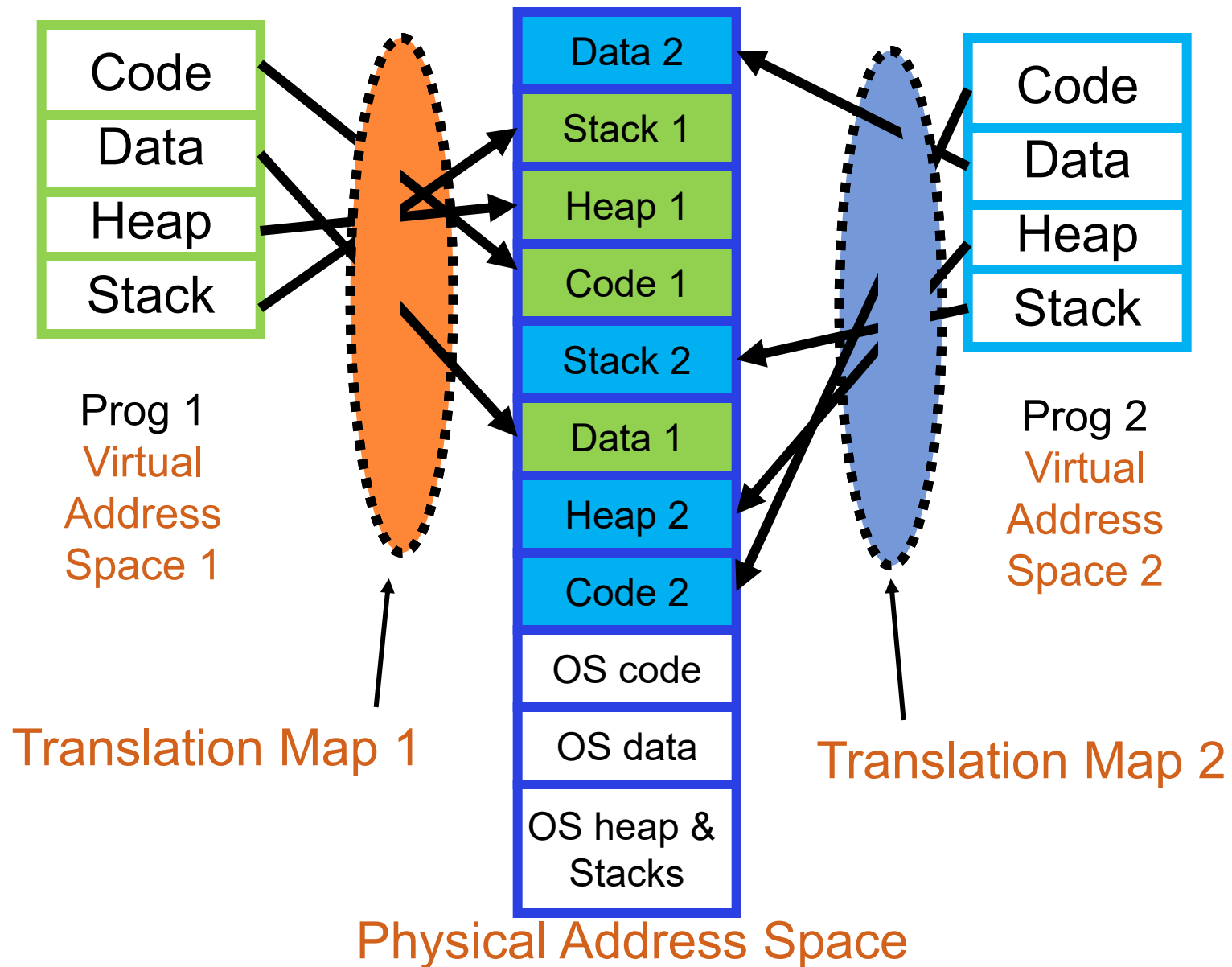
- Un singur BCP este activ la un moment dat
- Multiplexarea în timp a procesorului
- Acces controlat la resurse (altele decât procesorul)
 - Modurile utilizator / kernel
 - Maparea memoriei
 - Fiecare proces cu propriul spațiu de adrese



Stallings, pg. 134

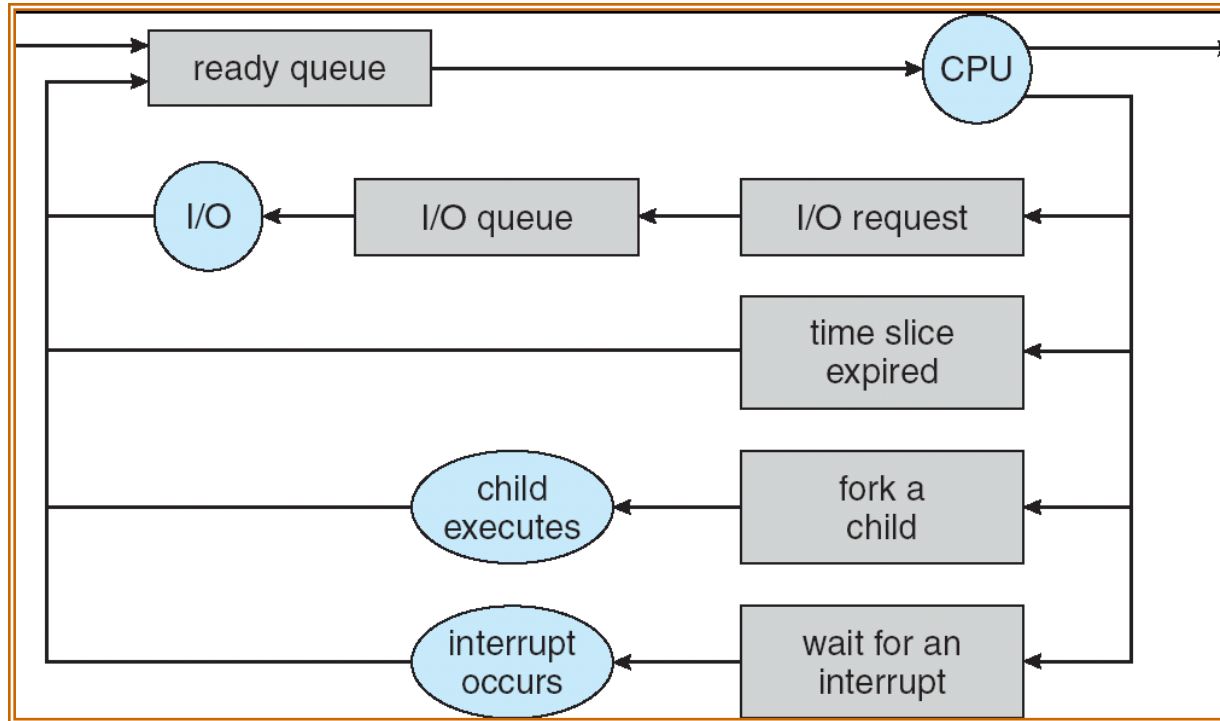
Figure 3.14 Process List Structures

SPAȚII DE ADRESE SEPARATE (VIRTUAL)



PLANIFICAREA PROCESELOR

Silberschatz, pg. 108

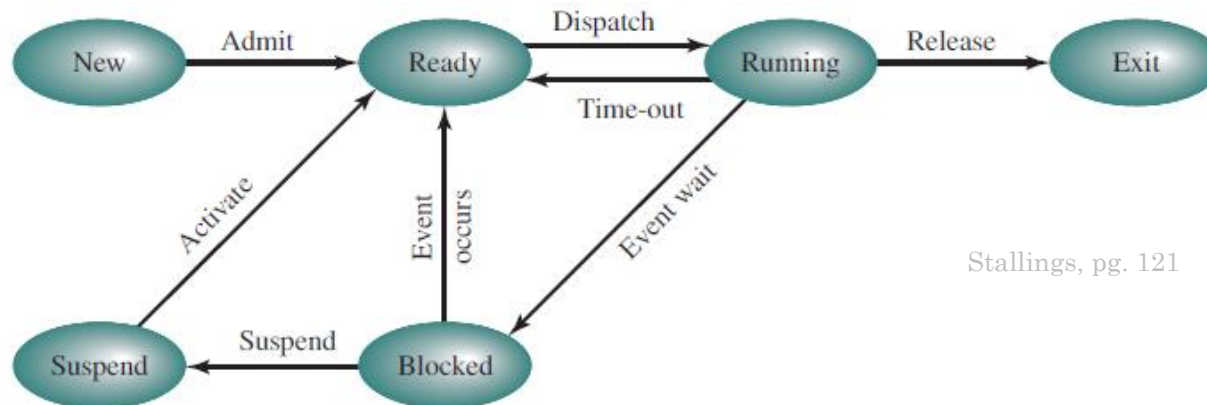


- Mai multe procese pot aștepta după același tip de eveniment
 - Se utilizează diferite cozi
- BCP sunt mutate din coadă în coadă în momentul schimbării de stare
 - Decizia privind următorul proces selectat pentru rulare ține de planificator
 - Există o mulțime de algoritmi de planificare

STĂRILE PROCESULUI CU PROCES SUSPENDAT

○ Procesul suspendat

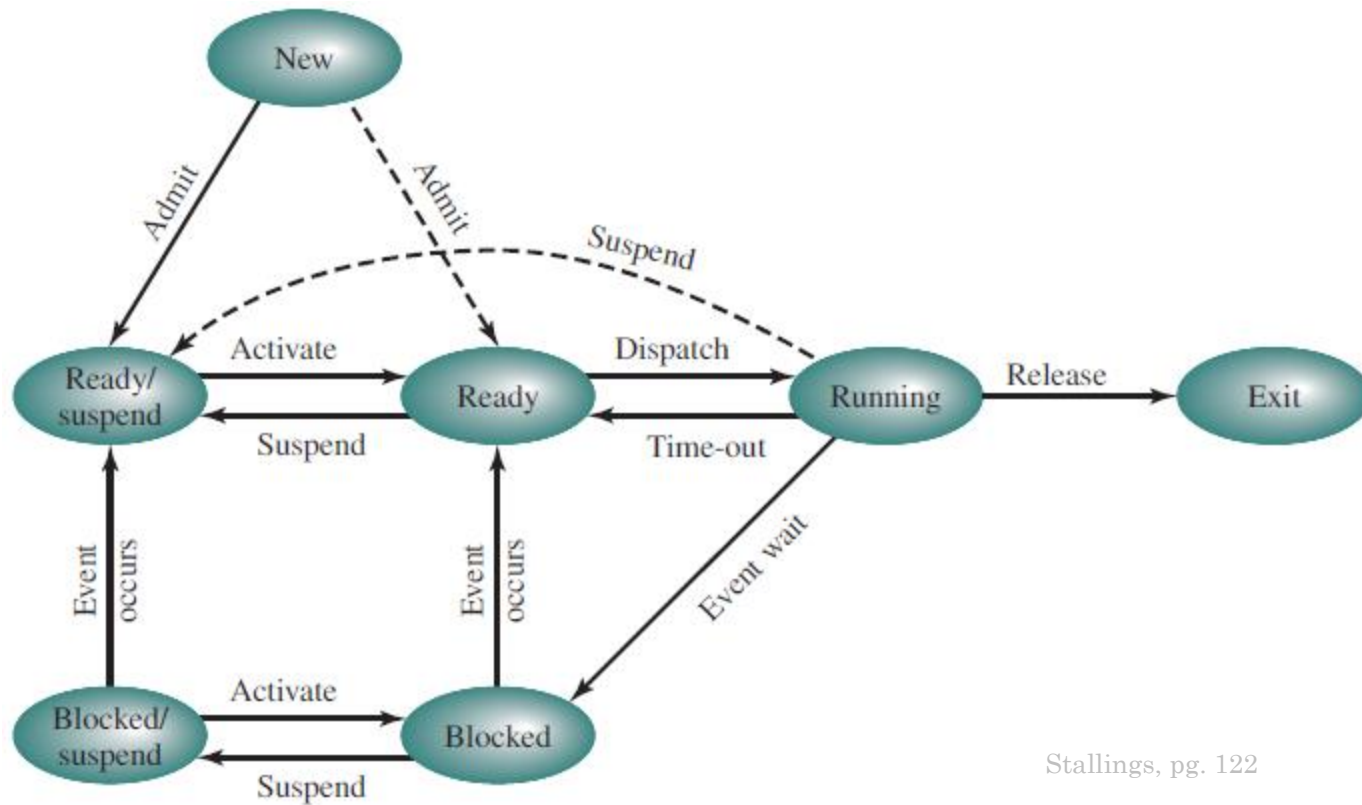
- Cu ajutorul memoriei virtuale (vezi cursurile următoare)
- Mutarea procesului (în întregime sau parțial) din memoria internă pe disc
- Pentru a rula mai multe (sau mai mari) procese decât încap în memorie



Stallings, pg. 121

(a) With one suspend state

STĂRILE PROCESULUI CU PROCES SUSPENDAT



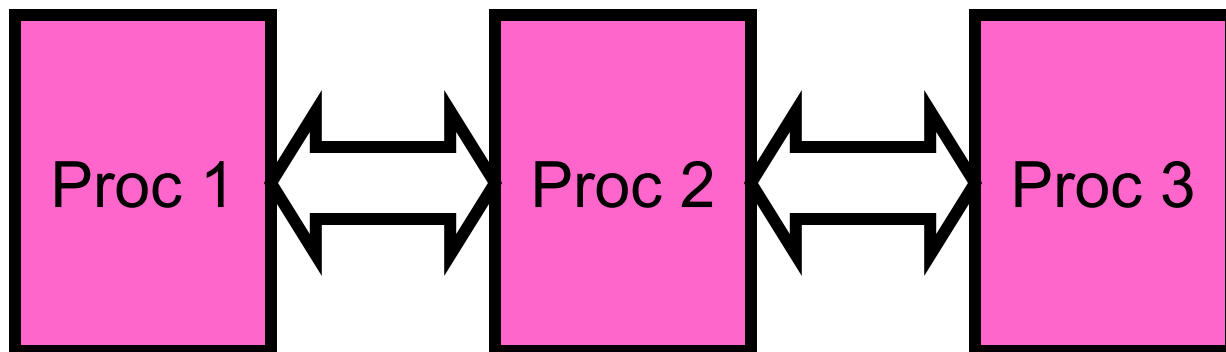
Stallings, pg. 122

(b) With two suspend states

RELAȚIA DINTRE PROCESE

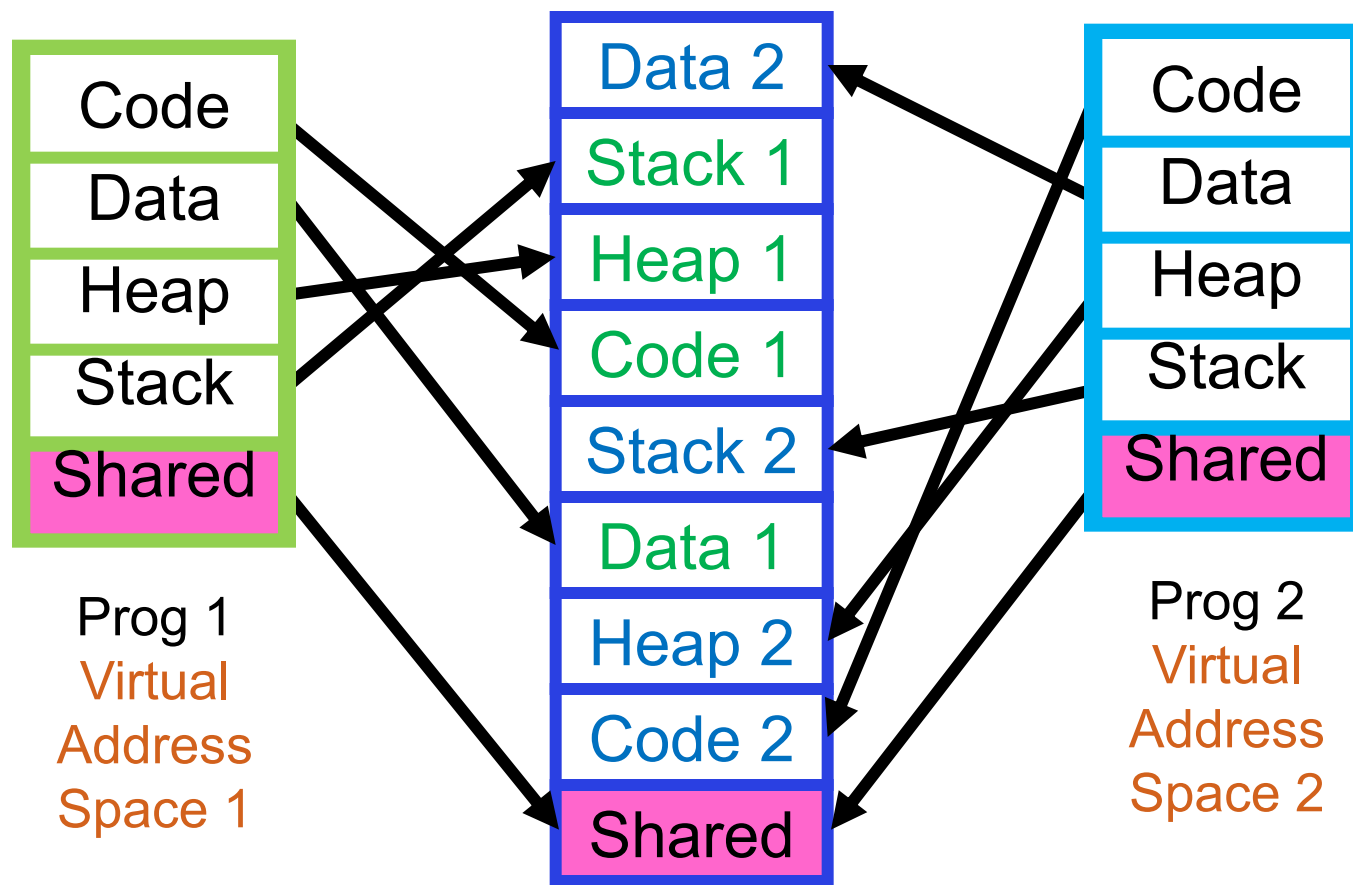
- Procese **independente**
 - Resursele cu care interacționează sunt disjuncte
- Procese **concurente**
 - Partajează resurse dar nu interschimbă informații
- Procese **cooperante**
 - Colaborează prin partajarea unor date sau prin schimb de mesaje

COLABORAREA ÎNTRE PROCESE



- Comunicarea între procese
 - Necesită mecanisme explicite
 - Motiv
 - Izolarea dată de spații de adrese separate (protecție)
- IPC – Inter Process Communication
 - Mecanisme de comunicare și sincronizare între procese
 - Comunicare prin memorie partajată
 - Comunicare prin transmitere de mesaje (cozi de mesaje)

COMUNICARE PRIN MEMORIE PARTAJATĂ



- Comunicarea se efectuează prin citirea / scrierea unui spațiu de memorie partajată
 - Comunicare fără încărcare semnificativă
 - Introduce probleme grele de sincronizare

CE PRESUPUNE CREAREA UNUI NOU PROCES?

- Crearea unui nou BCP (bloc de control al procesului)
 - Cost redus
- Crearea mapărilor de pagini pentru spațiul de adrese
 - Cost mediu
- Copierea datelor din părinte
 - Procesul fiu este copia părintelui
 - Inițial foarte costisitor
 - Soluție COW – Copy on Write
- Copierea stării i/e
 - Descriptori pentru fișierele deschise, etc.
 - Cost mediu

APELURI SISTEM - LINUX

- Identificarea procesului
- Crearea unui proces
- Familia de apeluri **exec**
- Terminarea procesului
- Așteptarea unui proces fiu

IDENTIFICAREA PROCESULUI

```
#include <unistd.h>

pid_t getpid(void);
pid_t getppid(void) ;
```

○ Exemplu

```
#include <stdio.h>
#include <unistd.h>
```

```
I am process 6316
My parent is 1816
```

```
int main(void) {

    printf("I am process %ld \n", getpid());
    printf("My parent is %ld \n", getppid());

    return 0;
}
```

CREAREA UNUI PROCES - FORK

```
#include <unistd.h>
```

```
pid_t fork(void);
```

- Procesul fiu are conținut identic cu procesul părinte
- Părinte și fiu – procese independente
 - Planificate independent
 - După apelul fork oricare din variantele următoare este posibilă:
 - CONTROLUL RĂMÂNE ÎN PROCESUL PĂRINTE
 - PROCESUL FIU ESTE PLANIFICAT – PREIA CONTROLUL
 - UN ALT PROCES ESTE PLANIFICAT
- Fork returnează
 - Pid-ul fiului în procesul părinte
 - 0 – în procesul fiu
- Procesele părinte și fiu continuă execuția concurentă de la instrucțiunea imediat următoare fork-ului

CREAREA UNUI PROCES - FORK

○ Exemplu simplu

```
#include <stdio.h>
#include <unistd.h>
```

```
int main(void) {
    int a = 1;

    printf("Before fork: I am process %ld \n", getpid());
    printf("Before fork: a = %d \n", a);

    fork();
    a++;
    printf("Both processes continue from here: I am process %ld \n", getpid());
    printf("In process %ld &a = %p, a = %d \n", getpid(), &a, a);

    return 0;
}
```

```
Before fork: I am process 1936
Before fork: a = 1
Both processes continue from here: I am process 11936
In process 1936 &a = 0xffffcbfc, a = 2
Both processes continue from here: I am process 9948
In process 9948 &a = 0xffffcbfc, a = 2
```

CREAREA UNUI PROCES - FORK

○ Exemplu cu testarea valorii returnate de fork

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
```

```
int main(void) {
    pid_t pid;
    int a = 1;

    printf("Before fork: I am process %ld \n", getpid());
    printf("Before fork: a = %d \n", a);

    pid = fork();
    switch (pid) {
        case -1: perror("Eroare creare proces"); exit(1);
        case 0: a = 5;
                printf("Child process: I am process %ld \n", getpid());
                printf("Child process: &a = %p, a = %d \n", &a, a);
                break;
        default:
            a = 10;
            printf("Parent process: I am process %ld \n", getpid());
            printf("Parent process: &a = %p, a = %d \n", &a, a);
    }
    return 0;
}
```

```
Before fork: I am process 4704
Before fork: a = 1
Parent process: I am process 4704
Parent process: &a = 0xffffcc08, a = 10
Child process: I am process 4660
Child process: &a = 0xffffcc08, a = 5
```

FAMILIA DE APELURI EXEC

- Apeluri pentru încărcarea unui nou cod în procesul apelant
 - Înlocuirea conținutului (alt cod), dar nu și a identității (pid, ppid)
- Pași separați fork și exec
 - Părintele are control asupra fiului înainte de exec (redirectări, etc.)
- Specificarea parametrilor se face în mod similar cu argumentele comenzilor din linia de comandă
- Clasificare
 - `execv`, `execvp`: număr variabil de argumente
 - `execl`, `execvp`: număr fix (prestabilit) de argumente
- Argumente
 - Primul arg. este calea către fișierul executabil
 - Restul argumentelor descriu comanda cu posibilele opțiuni, pornind de la numele comenzii

FAMILIA DE APELURI EXEC

○ Exemplu execl

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main(void) {
    pid_t childpid;

    childpid = fork();
    if (childpid == -1) {
        perror("Failed to fork");
        return 1;
    }
    if (childpid == 0) {
        execl("/bin/ls", "ls", "-l", NULL);
        perror("Child failed to exec ls");
        return 1;
    }
    if (childpid != wait(NULL)) {
        perror("Parent failed to wait due to signal or error");
        return 1;
    }
    return 0;
}
```

FAMILIA DE APELURI EXEC

○ Exemplu execl și execlp

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main(void) {
    pid_t childpid;

    childpid = fork();
    if (childpid == -1) {
        perror("Failed to fork");
        return 1;
    }
    if (childpid == 0) {
        execl("/bin/ls", "ls", "-l", NULL);
        perror("Child failed to exec ls");
        return 1;
    }
    if (childpid != wait(NULL)) {
        perror("Parent failed to wait due to signal or error");
        return 1;
    }
    return 0;
}
```

// the path is search by the system in
// directories stored in PATH variable
execlp("ls", "ls", "-l", 0);

FAMILIA DE APELURI EXEC

○ Exemplu execvp

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main()
{
    pid_t child_pid;
    /* Duplicate this process. */
    child_pid = fork();

    if (child_pid == 0) { // in child process
        /* The argument list to pass to the "ls" command. */
        char* arg_list[] = {
            "ls", /* argv[0], the name of the program. */
            "ls",
            "-l",
            "/",
            NULL }; /* list must end with a NULL. */

        execvp(arg_list[0], arg_list + 1);
        /* execvp returns only if an error occurs. */
        perror("an error occurred in execvp");
        abort();
    }
    else if (child_pid > 0) { // in parent process
        wait(NULL);
    }
    else { perror("an error occurred in fork");
        abort();
    }

    printf("done with main program\n");
    return 0;
}
```

TERMINAREA PROCESULUI

○ Apelul `exit`

- Terminarea voluntară a procesului apelant
- Se specifică un cod de terminare
 - 0 este considerat ca și cod de succes
 - Orice altceva se consideră terminare cu eroare
- Codul de exit este menținut până când procesul părinte va interoga modul de terminare a fiului (wait)
- Exemple
 - `exit(0);`
 - `exit(1);`

AȘTEPTAREA UNUI PROCES FIU

- Procesele părinte și fiu sunt concurente
- Nu știm care începe primul execuția (planificator)
 - La rulări succesive se poate modifica ordinea
- Sincronizare
 - Procesul părinte așteaptă terminarea fiului
 - Așteptarea terminării cu apelurile **wait** și **waitpid**

```
#include <sys/wait.h>
```

```
pid_t wait(int *stat_loc);
```

```
pid_t waitpid(pid_t pid, int *stat_loc, int options);
```

AȘTEPTAREA UNUI PROCES FIU

○ Exemplu

```
pid_t childpid;

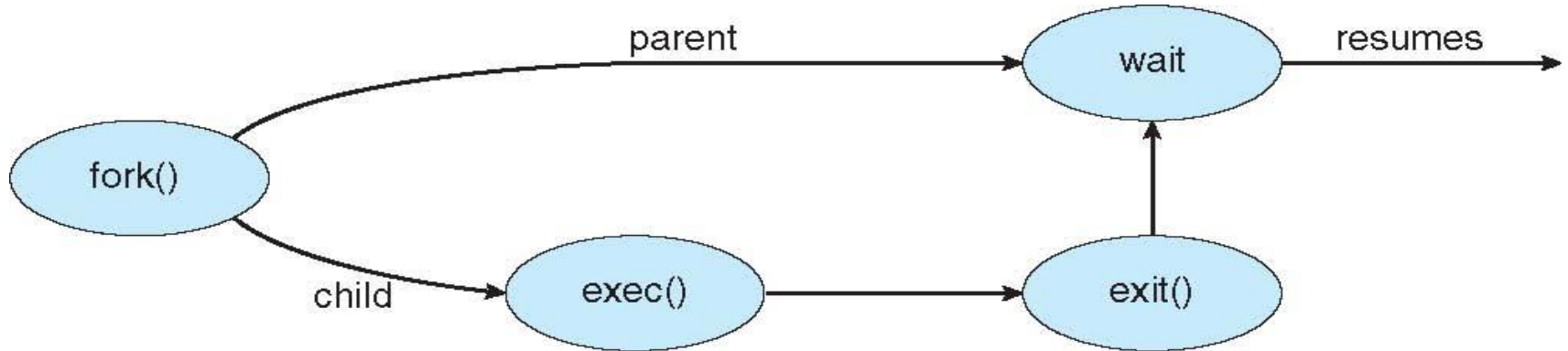
childpid = wait(NULL);
if (childpid != -1)
    printf("Waited for child with pid %ld\n", childpid);

int child_status;

wait(&child_status);

printf("Child process terminated with code %d\n",
       WEXITSTATUS(status));
```

APELURI SISTEM



Silberschatz, pg. 113

PROCESE ORFANE ȘI PROCESE ZOMBIE

○ exit și wait

- Apeluri care permit sincronizarea între părinte și fiu
- Furnizează un mod simplu de comunicare

• Sfârșitul este inevitabil

- Dar contează când survine

○ Procese orfane

- Procese a căror părinte se termină fără să aștepte după fii
- Procesele fii sunt adoptate de procesul init

○ Procese zombie

- Procesul fiu se termină dar părintele nu apelează wait
- Starea de terminare se menține de către SO până când părintele apelează wait sau se termină